

DoS Attack in CAN Bus

Yangala Reddy Sekhar (B231346CS)

Bayanaboina Venkata Subbaiah (B230871CS)

Guided by

Dr. Arun Raj Kumar P
Associate Professor, CSED

Department of Computer Science and Engineering
National Institute of Technology Calicut

May 8, 2026

Outline

- 1 Introduction
- 2 Problem Statement
- 3 Motivation
- 4 Existing Solutions
- 5 Research Gaps
- 6 Models Used
- 7 Features
- 8 Graph Construction
- 9 Training Process
- 10 Results
- 11 Performance Analysis

Introduction to Domain - Autonomous Vehicles

- Autonomous Vehicles (AVs) can sense environment and drive with minimal or no human intervention
- Improves safety and reduces human errors
- Classified into SAE Levels (0–5)

Level	<i>Automation</i>	<i>Steering Cruise</i>	<i>Environ- ment Monitoring</i>	<i>Fall- back Control</i>	<i>Driving Modes</i>
0	Non	H	H	H	N/A
1	Supportive	H,S	H	H	Some
2	Partial	S	H	H	Some
3	Conditional	S	S	H	Some
4	High	S	S	H	Some
5	Full	S	S	S	All

(S: SYSTEM, H: HUMAN)

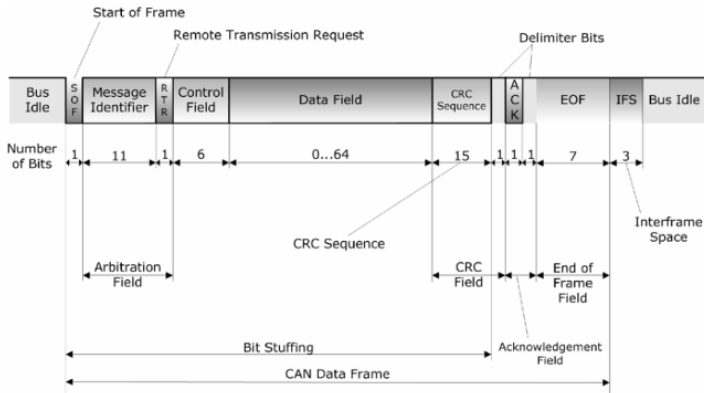
Source: ICEICT 2020, IEEE 3rd International Conference on Electronic Information and Communication Technology,

Security for Autonomous Vehicle Networks

Controller Area Networks

- Vehicles use internal communication networks
- CAN (Controller Area Network) is the most widely used
- Connects multiple Electronic Control Units (ECUs)
- Used in braking, steering, engine control and sensors

CAN Data Frame



Source: *A Comprehensive Guide to Controller Area Network*, 2nd Edition, Wilfried Voss

Need for Network Security

- CAN protocol has no built-in security mechanisms
- No authentication or encryption
- Any ECU can inject malicious messages

Problem Statement: DoS Attack in CAN

- CAN follows priority-based message arbitration
- Attackers exploit this by injecting high-priority messages
- Leads to Denial-of-Service (DoS) attack

Motivation

- Modern vehicles are increasingly connected and autonomous
- Critical functions (braking, steering, engine) depend on CAN communication
- DoS attacks can block these messages and disrupt vehicle operation
- This can lead to serious safety risks and potential accidents

Motivation (Continued)

- Existing solutions focus mainly on attack detection
- Real-time mitigation is essential for safety-critical systems
- Difficult to deploy on resource-constrained ECUs

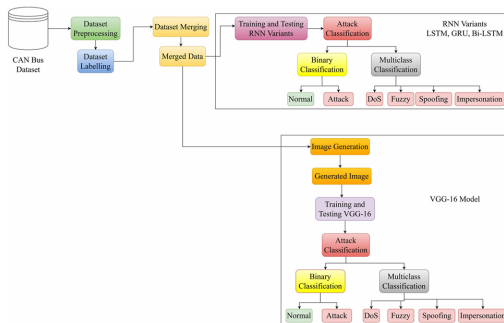
CNN & RNN Based IDS

- Deep learning IDS for CAN bus security
- Models used:
 - CNN (VGG-16) → spatial feature extraction
 - RNN (LSTM, GRU, Bi-LSTM) → temporal patterns
- Detects attacks using CAN message sequences and patterns

Source: Rai et al., Scientific Reports, 2025

Implementation

- Combined multiple datasets (Car Hacking, OTIDS, Survival)
- Feature extraction:
 - Temporal → RNN models
 - Spatial (image-based) → CNN (VGG-16)
- Train-test split: 80% / 20%



Performance (Car Hacking Dataset)

- **Binary Classification:**

- LSTM Accuracy: 99.89%
- VGG-16 Accuracy: 89.74%

- **Multiclass Classification:**

- VGG-16 Accuracy: up to 100% (DoS, RPM Spoofing)
- Overall high Precision, Recall, F1-score

Model	Metric	Combined			
		DoS	Impersonation	Malfunction	Benign
LSTM	Accuracy	0.92	1	0.05	1
	Precision	0.83	1	0.07	0.99
	Recall	0.06	1	1	0.48
	F1-Score	0.11	1	0.14	0.65
GRU	Accuracy	0.82	1	0.08	1
	Precision	0.83	1	0.07	0.99
	Recall	0.06	1	1	0.48
	F1-Score	0.11	1	0.14	0.65
Bi-LSTM	Accuracy	0.82	1	0.02	0.99
	Precision	0.83	1	0.07	0.99
	Recall	0.06	1	1	0.52
	F1-Score	0.11	1	0.52	0.67
VGG16	Accuracy	1	1	1	0.90
	Precision	1	1	1	1
	Recall	1	0.96	0.96	0.89
	F1-Score	1	0.98	0.98	0.94

CNN-LSTM-Attention IDS – Core Idea

Core Idea

- Learns attack patterns directly from CAN traffic
- CNN → extracts local payload features
- LSTM → captures temporal behavior
- Attention → focuses on important features

Input Features

- CAN ID
- Payload bytes
- Time-based features:
 - frequency, interval, repetition

Source: Rai et al., Scientific Reports 2025

Results

- Accuracy: 97.6%
- High detection for DoS, spoofing, replay

Practical Limits (adjustable)

- Large model → not suitable for ECU
- Trained on limited dataset
- No direct mitigation

Source: Rai et al., Scientific Reports 2025

Research Gap: Limitations of Deep Learning Models

- DL models (CNN, LSTM) have high parameter complexity
- High computation and memory requirements
- Not suitable for resource-constrained ECUs
- High latency → limits real-time use

Research Gap: Need for Practical IDS

- Real-world IDS requires:
 - Low latency
 - Low resource usage
 - High reliability
- Pure DL approaches fail to satisfy these constraints
- Limited work on:
 - Lightweight real-time models
 - Detection + mitigation integration
 - Hybrid approaches combining multiple techniques

Models Used

- Two models:
 - Gaussian Naive Bayes (GNB)
 - XGBoost Classifier
- Both models trained on same feature set
- Compared performance across window sizes
- Observed similar performance for both models

Feature Overview

- Total Features: 13
- Combination of:
 - 4 Statistical Features
 - 9 Graph-based Features
- Features extracted per window

Statistical Features

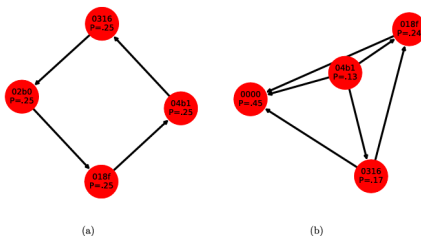
- Mean (frequency distribution of CAN IDs)
- Standard Deviation
- Entropy (randomness in CAN traffic)
- Score (threshold-based anomaly indicator)
- Normality-based scaling using reference values

Graph Features

- Graph constructed from CAN ID sequence
- Features extracted:
 - Number of Nodes
 - Number of Edges
 - Max / Min Indegree
 - Max / Min Outdegree
 - PageRank Features:
 - Max PageRank
 - Min PageRank
 - Median PageRank

Graph Construction

- Directed graph using CAN ID sequence
- Each CAN ID = node
- Edge formed between consecutive messages
- PageRank used to capture node importance



Source: GGNB (Islam et al.)

Feature Extraction Pipeline

- Sliding window approach
- Window sizes tested:
 - 50, 100, 500, 1000
- For each window:
 - Compute statistical features
 - Build graph and extract graph features
- Final feature vector = 13 features

Parameter Tuning

- Gamma thresholds used:
 - $\gamma_1, \gamma_2, \gamma_3$
- Tuned based on:
 - Normal reference dataset
 - Distribution of mean, std, entropy
- Thresholds tuned from range 1 to 5.

Training Process

- Dataset split:
 - 70% Training
 - 30% Testing
- K-Fold Cross Validation ($k=5$)
- Prevents overfitting
- Final model trained on full training data

Results (GNB - Window = 100)

```
[RESULT] Confusion Matrix:
```

```
[[9598   1]
 [  10 891]]
```

```
[RESULT] Classification Report:
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	9599
1	1.00	0.99	0.99	901
accuracy			1.00	10500
macro avg	1.00	0.99	1.00	10500
weighted avg	1.00	1.00	1.00	10500

- Near-perfect classification with very few misclassifications
- High precision and recall for both classes

Results (XGBoost - Window = 100)

```
[9597 2]
[ 7 894]]
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	9599
1	1.00	0.99	0.99	901
accuracy			1.00	10500
macro avg	1.00	1.00	1.00	10500
weighted avg	1.00	1.00	1.00	10500

- Performance comparable to GNB
- Slight improvement in detecting attack samples

Performance Analysis

- Both models achieved near-identical performance
- Very low false positives and false negatives
- Window size = 100 provides best trade-off between accuracy and computation
- Graph + statistical features improve separability of attacks

VCAN Setup & Testing

- Virtual CAN (VCAN) used for real-time testing on Linux
- Setup steps:
 - Create virtual environment (venv)
 - Enable VCAN interface:

VCAN Commands

```
sudo modprobe vcan  
sudo ip link add dev vcan0 type vcan  
sudo ip link set up vcan0
```

- traffic.py → generates CAN traffic
- ids.py → performs real-time intrusion detection

VCAN Results & Observations

- Random traffic → poor detection
- Replayed Car Hacking dataset on VCAN
- Achieved results similar to offline testing

```
===== CONFUSION MATRIX =====
```

```
[[1084    0]
 [    4  912]]
```

```
===== CLASSIFICATION REPORT =====
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1084
1	1.00	1.00	1.00	916
accuracy			1.00	2000
macro avg	1.00	1.00	1.00	2000
weighted avg	1.00	1.00	1.00	2000

```
=====
```

```
(can_env) sekhar@stud:~/can_ids_project$ █
```

References

- Rai et al., Securing the CAN Bus Using Deep Learning for Intrusion Detection in Vehicles, Scientific Reports, 2025.
- Groza et al., CANARY: A Reactive Defense Mechanism for Controller Area Networks, USENIX Security Symposium, 2021.
- Riadul Islam et al., GGNB: Graph-based Gaussian Naive Bayes Intrusion Detection System for CAN Bus.
- Junaid Khan et al., Intrusion Detection System for CAN-Bus Based on Statistical Characteristics of Attacks.
- Car Hacking Dataset (HCRL, 2018).
- Opel Astra Dataset (Real-world CAN dataset).